

Project 2: Schema Linking with Supervised Fine-Tuning

CSE/DSC 234: Data-Centric AI and AI Engineering

Sukruth Rao PID: A69042450
University of California San Diego

1 Training Data Methodology

1.1 Dataset Source

The training set consists of 301 examples from the SNAILS benchmark which was released with the project. Each example is a JSON object with fields `question_id`, `db_id`, `question`, `gold_sql`, and `schema_links`. The `schema_links` field holds the gold labels and was generated from the gold SQL using the provided `sql_to_schema_links.py` script. This extracts all tables and columns referenced in the query.

1.2 Schema Serialization

The `schemas` folder contains the database schemas that are used as part of each model’s input. Each `db_id` corresponds to one of the 17 JSON schema files in the folder. These files describe the database structure, including table names, column names, data types, primary keys, and foreign key relationships.

We implemented three schema serialization formats, with the chosen format kept consistent between training and evaluation for each model:

- **basic**: A simple table-to-columns dictionary using the schema’s original column order, without additional annotations.
- **pkfk**: The same dictionary, but with columns annotated using key information from `primary_keys` and `foreign_keys`. Columns may be marked as (PK), (FK→Table), or (PK,FK→Table). The model is instructed to output only the original column names without annotations.
- **sorted**: The tables are sorted alphabetically, and columns within each table are also sorted alphabetically, allowing us to test sensitivity to ordering.

The training and inference code use the same schema format for each setting, so models are evaluated using the same serialization that they were previously trained on. We use two types of inferences: (1) **one-shot** and (2) **two-stage**. During one-shot inference, the full schema serialization is exactly the same. In two-stage inference, the schema is first reduced/pruned to a smaller set of candidate tables before column selection, so the second-stage prompt uses only the pruned schema instead of the full annotated or sorted version. Because the schema is rebuilt without re-applying annotations or sorting, the **pkfk** and **sorted** formats are only fully preserved in the one-shot setting. Finally, the model’s JSON output is checked against the full schema, and any tables or columns not present in it are removed before scoring.

1.3 Output Format

Each model outputs a single JSON object that maps table names to lists of relevant column names. We then parse this output in `main.py`. We do this by first trying a direct JSON parse, and if that fails, we extract the first valid brace-delimited JSON object from the text. We then apply a `JSON-repair` when needed.

Eventually, after parsing we validate the prediction against the full database schema (case-insensitively) and remove any tables or columns that are not in the schema. This ensures that only valid schema links are kept. Finally, predictions are saved as a list of `{question_id, schema_links}` records, matching the evaluation format.

1.4 Augmentation

The original training data was unevenly distributed across databases, which is dominated by a few large databases, while the nine SBODemoUS sub-modules have only around 6 examples each (12 for SBODemoUS Human Resources). Hence, we believed that the model needed more training examples from them to produce better results. We therefore augmented the training set with synthetic question-SQL pairs for the underrepresented databases, where we used `Claude Code` to the questions and matching SQL pairs. We did not let the assistant write the gold labels, rather, every query was passed through the provided `sql_to_schema_links.py` script (through `generate_augmented_train.py`).

To validate the labels, we ran the extraction script (`sql_to_schema_links.py` again on all 159 augmented examples and confirmed that the generated `schema_links` matched the stored labels exactly. We also confirmed that no tables were linked to missing or empty column lists. This verification, along with the generation of the synthetic question/SQL pairs and the augmentation script itself, was carried out with the assistance of `Claude Code`. All added questions and SQL queries are synthetic and were created only from schemas already included in the released dataset.

2 Final Pipeline Architecture

We use the augmented training data (460 examples) in our final pipeline.

Base Model: We use a single instruction-tuned base model in `Qwen/Qwen2.5-1.5B-Instruct`. The model parameters are never updated during training.

Adapter: We train a standard LoRA adapter. The final configuration uses rank $r = 4$, $\alpha = 8$, dropout = 0.1, no bias term, and targets the attention projection layers `q_proj`, `k_proj`, `v_proj`, and `o_proj`. We use a learning rate of 10^{-5} , cosine scheduling, and train for 3 epochs. At inference time, the trained adapter is loaded on top of the base model using PEFT.

Prompt Template: Each query follows the Qwen chat format, which consists of a system and a user message. The system message is “*You are a schema-linking assistant. Given a question and a database schema, return ONLY a valid JSON object that maps table names to relevant column-name lists.*”. The user message is

```
Database schema: {schema}
Question: {question}
Return a JSON object with only the relevant tables as keys and lists of relevant
column names as values. You MUST include specific column names --- do not return
```

empty lists unless a table has no relevant columns. Example: `{"Orders": ["order_id", "total"], "Customers": ["name"]}`

The same prompt template is used during both training and inference.

Output decoding: Generation is fully deterministic, using greedy decoding(`do_sample=False`) and with a fixed seed and a maximum of 256 new tokens.

Post-Processing: We convert the model’s raw output into a valid JSON object. We do so by first trying to parse it directly as JSON. If that fails, we extract the first complete JSON-like block from the text, and apply JSON-repair to fix formatting issues if required.

After parsing, we check the prediction against the full database schema with case-insensitivity. We remove any table or column that does not exist in the schema. We then save the results as `question_id`, `schema_links` records for evaluation using `eval.py`

3 Experimentation Methodology

Run	Fmt	r/α	LR	Sched	Len	Tgt	Infer
Stage 1: experkrj_40 — Qwen2.5-1.5B-Instruct, original 301-ex corpus, one-shot							
1	basic	8/16	1×10^{-4}	linear	120 st	all	1s
2	pkfk	8/16	1×10^{-4}	linear	120 st	all	1s
3	sorted	8/16	1×10^{-4}	linear	120 st	all	1s
4	basic	16/32	1×10^{-4}	linear	120 st	all	1s
5	basic	32/64	1×10^{-4}	linear	120 st	all	1s
6	basic	8/16	2×10^{-4}	linear	120 st	all	1s
7	basic	8/16	5×10^{-5}	linear	120 st	all	1s
8	basic	16/32	1×10^{-4}	linear	120 st	attn	1s
9	pkfk	16/32	1×10^{-4}	linear	150 st	all	1s
10	pkfk	8/16	1×10^{-4}	linear	150 st	all	1s
11	basic	16/32	1×10^{-4}	linear	150 st	all	1s
12	basic	4/8	2×10^{-5}	linear	120 st	attn	1s
13	basic	8/16	1×10^{-4}	linear	200 st	all	1s
14	pkfk	16/32	1×10^{-4}	linear	200 st	all	1s
Stage 2: augmented_final_experiments_rf — Qwen2.5-1.5B-Instruct, augmented 460-ex corpus, two-stage							
1	basic	4/8	1×10^{-5}	linear	2 ep	attn	2s
2	basic	4/8	1×10^{-5}	linear	3 ep	attn	2s
3	basic	4/8	5×10^{-6}	linear	2 ep	attn	2s
4	basic	4/8	2×10^{-5}	linear	2 ep	attn	2s
5	basic	4/8	1×10^{-5}	cosine	3 ep	attn	2s
6	pkfk	4/8	1×10^{-5}	linear	2 ep	attn	2s
7	sorted	4/8	1×10^{-5}	linear	2 ep	attn	2s
Stage 3: augmented_qwen3_1.7b_top3_experiment_rf — Qwen3-1.7B, augmented 460-ex corpus, two-stage							
1	basic	4/8	1×10^{-5}	cosine	3 ep	attn	2s
2	basic	4/8	1×10^{-5}	linear	2 ep	attn	2s
3	basic	4/8	1×10^{-5}	linear	3 ep	attn	2s
Stage 4: augmented_smollm2_top3_experiment_rf — SmolLM2-1.7B-Instruct, augmented 460-ex corpus, two-stage							

(Table 1 continued)

Run	Fmt	r/α	LR	Sched	Len	Tgt	Infer
1	basic	4/8	1×10^{-5}	cosine	3 ep	attn	2s
2	basic	4/8	1×10^{-5}	linear	2 ep	attn	2s
3	basic	4/8	1×10^{-5}	linear	3 ep	attn	2s

Table 1: All 27 fine-tuning configurations across the four experiment families, with their full hyperparameter knobs. Fmt: prompt-serialization format (basic / pkfk / sorted). r/α : LoRA rank and alpha. LR: learning rate. Sched: LR scheduler. Len: training length (epochs, or `max_steps` for `experkrj_40`). Tgt: LoRA target modules (`attn` = q/k/v/o_proj; `all` = attn + gate/up/down_proj). Infer: inference mode (1s = one-shot full schema; 2s = two-stage prune then select).

The experimentation was done in four stages, with increments being made with each stage. We tested across 27 different configurations and four experiment groups.

In stage 1, we did a broad search using the recommended Qwen2.5-1.5B-Instruct model trained on the original 301 examples. We tried different settings for LoRA size, learning rate, and training lengths (120, 150, 200 steps using `max_steps`). We also tried applying LoRA only to the attention layers or to both attention and MLP layers. The results varied a lot. Many runs performed poorly, but the best run achieved a score 0.4066. We found that the small LoRA sizes, attention tuning, and low learning rates worked best, while larger settings made it extremely unstable.

Stage 2 focused more on the best settings from Stage 1 and trained again using the augmented dataset containing 460 examples. Here, we switched to using 2 or 3 epochs by utilizing `num_train_epochs`. We also moved from one-shot inference to a two-stage process, where the model first selects relevant tables and then chooses columns from a reduced schema. We made extremely small adjustments to the learning rate, scheduler type, and prompt format. This stage gave the best result so far.

In Stage 3, we tested a larger model (Qwen3-1.7B) to see if size would help. We used the same best settings from Stage 2, but results were slightly worse which showed us that a bigger model alone did not improve performance.

Finally, in Stage 4, since the larger Qwen model had not helped, we suspected the Qwen family itself might be the holding the leaderboard score back, so we tried a different model family (SmolLM2-1.7B-Instruct) using the same top settings from Stage 2.

We will review our results and discussion in the next section.

4 Results and Discussion

We observe that our best configuration is the experiment that uses the `basic` format, $r = 4$, $\alpha = 8$, attention-only LoRA, learning rate = 10^{-5} , and two-stage inference. It achieved a leaderboard score of 0.4749, with table score = 0.5695 and column score = 0.3802.

We can see that from Stage 1 that small LoRA ranks and attention-only tuning are the most effective factors, and hence were considered into the subsequent experiments. The best run in Stage 1 scored 0.4066 with rank 4, and attention-only tuning. On the other hand, higher ranks performed much worse. For example, rank 8 with attention + MLP reached 0.3469, and higher ranks of 16 and 32 dropped below 0.25.

#	Fmt	r/α	LR	Sched	Train	Tgt	Infer	Table	Col	Lead
<i>augmented_final_experiments_rf</i> – Qwen2.5-1.5B-Instruct (augmented 460 eg corpus)										
1	basic	4/8	1e-5	linear	2 ep	attn	2-stage	0.5695	0.3786	0.4741
2	basic	4/8	1e-5	linear	3 ep	attn	2-stage	0.5574	0.3901	0.4738
3	basic	4/8	5e-6	linear	2 ep	attn	2-stage	0.5525	0.3700	0.4612
4	basic	4/8	2e-5	linear	2 ep	attn	2-stage	0.5581	0.3882	0.4732
5	basic	4/8	1e-5	cosine	3 ep	attn	2-stage	0.5695	0.3802	0.4749
6	pkfk	4/8	1e-5	linear	2 ep	attn	2-stage	0.4609	0.0409	0.2509
7	sorted	4/8	1e-5	linear	2 ep	attn	2-stage	0.2377	0.0452	0.1414
<i>augmented_qwen3_1.7b_top3_experiment_rf</i> – Qwen3-1.7B (augmented 460 eg corpus)										
1	basic	4/8	1e-5	cosine	3 ep	attn	2-stage	0.5350	0.3599	0.4474
2	basic	4/8	1e-5	linear	2 ep	attn	2-stage	0.5140	0.3593	0.4366
3	basic	4/8	1e-5	linear	3 ep	attn	2-stage	0.5409	0.3702	0.4555
<i>augmented_smollm2_top3_experiment_rf</i> – SmolLM2-1.7B-Instruct (augmented 460 eg corpus)										
1	basic	4/8	1e-5	cosine	3 ep	attn	2-stage	0.2568	0.1420	0.1994
2	basic	4/8	1e-5	linear	2 ep	attn	2-stage	0.2502	0.1491	0.1996
3	basic	4/8	1e-5	linear	3 ep	attn	2-stage	0.2735	0.1585	0.2160
<i>experkrj_40</i> – Qwen2.5-1.5B-Instruct (original 301 eg corpus, trained by max_steps)										
1	basic	8/16	1e-4	linear	120 st	all	1-shot	0.1800	0.1098	0.1449
2	pkfk	8/16	1e-4	linear	120 st	all	1-shot	0.2246	0.1343	0.1795
3	sorted	8/16	1e-4	linear	120 st	all	1-shot	0.2448	0.1279	0.1863
4	basic	16/32	1e-4	linear	120 st	all	1-shot	0.0495	0.0419	0.0457
5	basic	32/64	1e-4	linear	120 st	all	1-shot	0.0539	0.0323	0.0431
6	basic	8/16	2e-4	linear	120 st	all	1-shot	0.0542	0.0431	0.0486
7	basic	8/16	5e-5	linear	120 st	all	1-shot	0.4361	0.2577	0.3469
8	basic	16/32	1e-4	linear	120 st	attn	1-shot	0.3297	0.1579	0.2438
9	pkfk	16/32	1e-4	linear	150 st	all	1-shot	0.1328	0.0750	0.1039
10	pkfk	8/16	1e-4	linear	150 st	all	1-shot	0.2382	0.1586	0.1984
11	basic	16/32	1e-4	linear	150 st	all	1-shot	0.0462	0.0318	0.0390
12	basic	4/8	2e-5	linear	120 st	attn	1-shot	0.4978	0.3154	0.4066
13	basic	8/16	1e-4	linear	200 st	all	1-shot	0.0570	0.0579	0.0574
14	pkfk	16/32	1e-4	linear	200 st	all	1-shot	0.1557	0.0848	0.1202

Table 2: All 27 fine-tuning configurations across the four RapidFire experiment families. Hyperparameters: prompt format (basic / pkfk / sorted), LoRA rank/alpha (r/α), learning rate, scheduler, training length (epochs or `max_steps`), and LoRA target modules (*attn* = q/k/v/o_proj; *all* = attn + gate/up/down_proj). *Infer* is the inference mode: one-shot serializes the full schema in a single pass; two-stage prunes the schema, selects tables, then selects columns. Table/Col/Lead are the table, column, and leaderboard scores respectively. *experkrj_40* was trained on the original training data (301 examples) by step count; the other three families used the augmented 460-example corpus and were trained by epoch. The entry in bold is the final selected configuration.

Based on the results obtained from Stage 1, Stage 2 fixed the best settings: $r = 4$, $\alpha = 8$, and attention-only LoRA for all experiments. The improvement from 0.40 to 0.47 can also further be attributed to the augmented training set and the switch in inference type from one-shot to two-stage. The cosine and linear schedulers achieve identical performance, however we can see a noticeable improvement in table scores when using cosine schedulers. However, since all of these settings were changed together, we weren't able to pinpoint their individual effects on the leaderboard scores.

The `pkfk` and `sorted` formats perform worse than `basic` format in both table and column scores in a lot of cases. We speculate that this might be because the extra reordering and annotations to the inputs might be adding noise, which causes wrong outputs.

Scaling up the model also does not help. Using Qwen3-1.7B gives slightly lower scores than Qwen2.5-1.5B, and using SmolLM2-1.7B showed worse results.

Supprises: Using larger models did not help at all. Both Qwen3-1.7B, and SmolLM2-1.7B performed worse than the smaller model in Qwen2.5-1.5B.

We also didn't expect adding more structure to the input through `pkfk` and `sorted` formats would hurt the performance (drastically in some cases), but we suspect that it might be adding more noise.

What we'd try with more time/budget: We'd try to concentrate on improving the column score. We could try that by expanding the training dataset well beyond 460 examples and train for more epochs.

With more time, we would improve the training infrastructure since we lost some time because RapidFire AI occasionally hanged during experimentation, so a more stable setup such as saving checkpoints regularly could allow resuming from the last checkpoint and prevent the loss of progress.

5 AI Tools Statement

We used `Claude Code` extensively throughout this project to help us debug the full pipeline, including fixing issues with JSON parsing and repair broken or incomplete outputs, handling empty-list edge cases, and fixing prompt formatting bugs. We also used it to help write and improve the `.sh` scripts to run RapidFire AI experiments and inference in the background. It also suggested the idea of two-stage inference and helped debug it in the final implementation. All generated code was reviewed, understood, and iteratively modified. The experimentation code was also modified to fix bugs by the coding agent to align with our pipeline configurations and findings.

We used `claude-sonnet-4-6` for grammar corrections, paragraph organization, and writing suggestions throughout this report. It was not used to write entire sections of the report.

No AI tool was used to generate git commits to tamper commit history.